

YUVA INTERN

EMBEDDED SYSTEMS INTERNSHIP | WEEK 1 TECHNICAL REPORT

IoT-Based Smart Environmental Monitoring and Alert System

ESP32-based temperature, humidity and air-quality monitoring node with local alerts and Wi-Fi reporting

Week 1 Task

System Requirements Analysis and Block Diagram Design

Item	Details
Student Name	PILLA YASHWANTH
E-mail	Yaswanthvasu79@gmail.com
Institute	Ignite Embedded system
Internship Organization	YUVA INTERN
Report Date	07/10/2026
Project Status	Proposed / simulated embedded-system design. No physical hardware has been built or tested at this stage.

Table of Contents

1. Introduction	4
2. Problem Statement.....	4
3. Project Objectives	5
3.1 Overall Project Objective	5
3.2 Specific Objectives	5
3.3 Week 1 Boundary	5
4. System Overview	5
4.1 Practical Embedded-System Application	5
4.2 Intended Functionality.....	5
4.3 Performance Expectations.....	6
4.4 General Operating Sequence.....	6
4.5 Proposed Design Assumptions	6
5. Functional Requirements.....	7
6. Non-Functional Requirements.....	8
7. Input Specifications	8
7.1 Sensor Inputs	8
7.2 MQ-135 Signal Conditioning	9
7.3 Configurable Thresholds.....	9
8. Output Specifications	10
8.1 OLED Display	10
8.2 Alert Outputs	10
8.3 Wi-Fi Telemetry Output	10
8.4 Serial Diagnostic Output	10
9. Hardware Requirements.....	11
9.1 Proposed Pin Allocation.....	11
9.2 Physical Design Considerations	11
10. Software Requirements.....	12
11. Communication Interfaces	12
11.1 I2C (OLED)	12
11.2 GPIO (LED, buzzer and DHT22 data)	13
11.3 ADC (MQ-135).....	13
11.4 Wi-Fi (remote data transmission).....	13
11.5 UART and BLE.....	13
12. Power Management	13
12.1 Power Architecture.....	13
12.2 Preliminary Current Estimate	13
12.3 Power-Management Considerations.....	14
13. Real-Time and Performance Requirements.....	14
13.1 Analytical Timing Estimates	15

13.2 Design Targets versus Measured Results	15
14. Hardware Architecture	15
14.1 Interface and Driver Circuits (Proposed)	16
14.2 Voltage Levels	16
15. Software Architecture	16
15.1 Scheduling Approach	17
15.2 System State Model	17
15.3 Shared Data (Conceptual)	17
16. Detailed Block Diagram	17
16.1 Explanation of Each Block	18
16.2 Diagram Specification (for Redrawing)	19
17. Software Flowchart	19
17.1 Flowchart Notes	20
18. Data Flow	21
19. Component Selection and Design Justification	21
19.1 Why ESP32?	21
19.2 Why DHT22?	22
19.3 Why MQ-135?	22
19.4 Why an I2C OLED?	22
19.5 Why Wi-Fi?	22
19.6 Why LED and Buzzer?	22
19.7 Why a Modular Hardware and Software Architecture?	22
20. Error Handling and Fault Considerations	23
21. Reliability and Security Considerations	23
21.1 Reliability	23
21.2 Security	24
21.3 Safety Note	24
22. Requirement Traceability Matrix	24
23. Methodology	25
24. Limitations	26
25. Future Improvements	26
26. Week 1 Deliverables	27
26.1 Deliverables Produced	27
26.2 Foundation for the Following Weeks	27
27. Conclusion	27

1. Introduction

Temperature, humidity and air quality strongly influence comfort, health, equipment life and the condition of stored materials in indoor spaces such as laboratories, classrooms, workshops, storage rooms and homes. Embedded systems make it practical to monitor these quantities continuously at low cost: a microcontroller can sample sensors, interpret the data locally, signal abnormal conditions immediately and, where a network is available, report to a remote user.

This report documents Week 1 of a four-week Embedded Systems Internship. The project is an **IoT-Based Smart Environmental Monitoring and Alert System** built around an ESP32 microcontroller, a DHT22 temperature/humidity sensor, an MQ-135 gas sensor used as a prototype air-quality input, a 0.96-inch I2C OLED display, an LED and buzzer for local alerts, and the ESP32's built-in Wi-Fi for data transmission to a conceptual remote dashboard. The same project continues through Weeks 1 to 4.

The Week 1 task is **System Requirements Analysis and Block Diagram Design**. Its purpose is to plan the system before any firmware is written. Decisions made at this stage about requirements, interfaces, pin allocation, power supply, timing targets and software structure determine how much rework is needed in later weeks, so they are documented here in a form that can be reviewed, traced and reused.

Scope and honesty statement. This is a proposed and simulated design. No physical hardware has been assembled or tested, no firmware has been written, and no cloud service has been deployed. All timing, current and performance figures in this report are design targets or planning estimates, not measurements. Component characteristics are quoted as typical published values and must be verified against the datasheets of the exact parts before procurement.

The remainder of the report follows the sequence of an engineering specification: problem and objectives (Sections 2 to 4), requirements (Sections 5 to 13), architecture and diagrams (Sections 14 to 18), design justification and fault analysis (Sections 19 to 21), traceability and methodology (Sections 22 and 23), and finally limitations, future work, deliverables and conclusion (Sections 24 to 27).

2. Problem Statement

In many indoor environments, temperature, humidity and air quality are checked only occasionally, by hand, or not at all. This creates several practical problems:

- **No continuous record.** Short excursions, such as an air-conditioning failure overnight, go unnoticed until damage or discomfort has already occurred.
- **Delayed reaction.** Conditions such as high humidity (which promotes condensation and mould) or poor ventilation (accumulation of gases) develop gradually and are hard to notice without instruments.
- **Limited visibility.** Even where a local thermometer or hygrometer exists, the reading is only useful to a person standing next to it.
- **Cost and flexibility.** Commercial monitoring products are often expensive, closed, or not configurable for a specific room or threshold.

Problem statement. To design a low-cost, ESP32-based embedded monitoring node that periodically measures temperature, humidity and a gas-based air-quality indication; presents the results on a local OLED display; raises an immediate local alert through an LED and buzzer when configurable thresholds are exceeded; and transmits the data over Wi-Fi to a remote dashboard, while continuing local monitoring and alerting if Wi-Fi is unavailable.

Designing such a node involves more than connecting modules. The main engineering challenges that the requirements and architecture must address are:

- Low-cost sensors have limited accuracy, require warm-up and calibration, and can fail or return invalid data.

- Network connectivity is unreliable; Wi-Fi operations can take long or fail, and must not stop sensing or alerting.
- Different tasks have different timing needs (for example, the DHT22 cannot be sampled faster than about once every two seconds, while alerts should be prompt).
- The MQ-135 heater draws significantly more current than the other components and its analog output can exceed the ESP32 ADC input range, so the power and signal-conditioning design must be deliberate.
- The firmware must remain maintainable so that it can be developed, debugged and integrated in stages over the following weeks.

3. Project Objectives

3.1 Overall Project Objective

To develop, over four weeks, an ESP32-based environmental monitoring and alert node that senses temperature, humidity and air-quality indication, displays and evaluates the data locally, alerts users when thresholds are exceeded, and reports data over Wi-Fi to a remote dashboard.

3.2 Specific Objectives

ID	Objective
O-1	Identify a practical embedded-system application and define the intended functionality and performance expectations of the system.
O-2	Derive complete, numbered, testable functional and non-functional requirements.
O-3	Specify all inputs, outputs and communication interfaces, including how each connects to the ESP32.
O-4	Define a power architecture (5 V USB to 3.3 V) and estimate power distribution needs.
O-5	Set realistic real-time and performance design targets, clearly separated from measured results.
O-6	Design the hardware architecture and a detailed block diagram with labelled interfaces.
O-7	Design a modular software architecture, operating flowchart and state model without implementing firmware.
O-8	Analyse likely faults, reliability and security considerations, and record limitations honestly.
O-9	Produce a requirement traceability matrix that links requirements to components and modules, as a basis for Weeks 2 to 4.

3.3 Week 1 Boundary

Week 1 produces analysis and design documentation only. Firmware development belongs to Week 2, debugging and optimisation to Week 3, and system integration and final testing to Week 4. None of the later-week activities is claimed as completed in this report.

4. System Overview

4.1 Practical Embedded-System Application

The system is a stand-alone environmental monitoring node intended for indoor use. Representative application settings are a laboratory or workshop (comfort and equipment conditions), a storeroom (humidity-sensitive stock), a classroom or office (ventilation awareness) and a home (basic comfort and air-freshness indication). The node is a **monitoring and advisory device**: it informs people so that they can act (open a window, switch on cooling, inspect a room). It is not intended as a certified safety or gas-detection instrument.

4.2 Intended Functionality

- Read temperature and humidity from a DHT22 and a gas-sensor voltage from an MQ-135.
- Validate and process the readings in the ESP32 (range checks, averaging, unit conversion, air-quality classification).

- Show temperature, humidity, air-quality status, alert status and Wi-Fi status on a 0.96-inch OLED.
- Compare readings with configurable thresholds and drive an LED and buzzer when a threshold is exceeded.
- Connect to Wi-Fi and periodically transmit the readings and status to a conceptual remote dashboard.
- Continue sensing, display and alerting locally when Wi-Fi is unavailable, and reconnect automatically when it returns.

4.3 Performance Expectations

The system is expected to behave as a soft real-time monitor: environmental quantities change slowly, so sampling every one to two seconds is sufficient, but abnormal readings should lead to a local alert within a few seconds. Display updates are periodic, network transmission is periodic rather than continuous, and a network failure must never delay a local alert. Quantitative design targets are given in Section 13.

4.4 General Operating Sequence

Step	Operation	Description
1	Power ON	5 V USB supply is applied; the 3.3 V rail comes up and the ESP32 starts. Outputs are held in a safe (OFF) state.
2	Initialize ESP32	Configure clocks, GPIO directions, ADC attenuation, I2C bus, serial port and watchdog.
3	Initialize sensors	Prepare the DHT22 interface and the MQ-135 ADC channel; start the MQ-135 warm-up timer.
4	Initialize OLED	Initialise the I2C display and show a start-up message; record a display-fault flag if it does not respond.
5	Connect to Wi-Fi	Attempt connection with a timeout. Failure selects offline mode; monitoring still starts.
6	Read sensors	Acquire DHT22 and MQ-135 data on their scheduled periods.
7	Validate / process data	Reject invalid values, filter, convert and classify the air-quality reading.
8	Update OLED	Refresh displayed values and status indicators.
9	Check thresholds	Compare processed values against configured limits, applying hysteresis.
10	Activate / deactivate alert	Drive the LED and buzzer according to the alert state.
11	Transmit data	If the transmission interval has elapsed and Wi-Fi is connected, send a data message.
12	Wait and repeat	Wait for the next monitoring cycle using non-blocking timing, then return to step 6.

4.5 Proposed Design Assumptions

The following assumptions are made so that the requirements are concrete. They are proposals that may be revised in later weeks.

ID	Proposed assumption
A-01	The target board is a standard ESP32 development board (ESP32-WROOM-32 class module with USB-serial interface and on-board 3.3 V regulator).
A-02	Firmware will be written in C/C++ using the Arduino framework for ESP32 (Arduino IDE or PlatformIO). FreeRTOS, which underlies the framework, may be used where useful.
A-03	A 5 V USB power source rated at least 1 A is available; a 500 mA USB port may be insufficient (see Section 12).
A-04	A 2.4 GHz WPA2-protected Wi-Fi network is available but may be intermittent.
A-05	The remote dashboard is conceptual. The device is assumed to send JSON-formatted data using HTTP(S) or MQTT; the final protocol is chosen in Week 2.
A-06	The node is used indoors in a non-condensing environment, with a proposed ambient operating range of roughly 0 to 50 degrees C.
A-07	One node monitors one room. Multiple nodes are a future extension identified by a device ID.
A-08	The MQ-135 is used for a relative, qualitative air-quality indication, not for absolute gas concentration in ppm.
A-09	Default alert thresholds are initial engineering proposals and will be tuned; they are not derived from experiments.

5. Functional Requirements

Requirements use the word **shall** for mandatory behaviour and **should** for desirable behaviour. Each requirement is uniquely numbered so that it can be traced to a component or module (Section 22) and verified in later weeks.

Table 1: Functional requirements

ID	Requirement	Priority	Primary module
FR-01	The system shall initialise the required hardware (GPIO, ADC, I2C, serial, Wi-Fi radio) during start-up.	Shall	System Init
FR-02	The system shall acquire temperature readings from the DHT22.	Shall	Sensor Acquisition
FR-03	The system shall acquire humidity readings from the DHT22.	Shall	Sensor Acquisition
FR-04	The system shall acquire air-quality sensor readings from the MQ-135 through the ESP32 ADC.	Shall	Sensor Acquisition
FR-05	The ESP32 shall process sensor data (unit conversion, averaging/filtering, classification).	Shall	Data Processing
FR-06	The system shall validate each reading (missing, out-of-range or implausible values) before it is displayed, evaluated or transmitted.	Shall	Data Processing
FR-07	The system shall allow the MQ-135 a warm-up period and shall not issue air-quality alerts until the period has elapsed.	Shall	Sensor Acquisition / Threshold Monitor
FR-08	The system shall display temperature and humidity on the OLED with units.	Shall	Display Manager
FR-09	The system shall display an air-quality status category (for example GOOD, MODERATE, POOR, WARMING UP).	Shall	Display Manager
FR-10	The system shall display alert status and Wi-Fi status on the OLED.	Shall	Display Manager
FR-11	The system shall compare readings against configurable thresholds.	Shall	Threshold Monitor
FR-12	The system shall apply hysteresis (and/or confirmation over consecutive samples) so that alerts do not chatter near a threshold.	Shall	Threshold Monitor
FR-13	The system shall activate the LED and buzzer when a threshold is exceeded.	Shall	Alert Manager
FR-14	The system shall deactivate the alert when readings return to the normal range.	Shall	Alert Manager
FR-15	The system should use distinguishable alert patterns (for example steady LED for alert, slow blink for fault).	Should	Alert Manager
FR-16	The ESP32 shall establish Wi-Fi communication using configured network credentials.	Shall	Wi-Fi Communication
FR-17	The system shall transmit monitoring data through Wi-Fi at a periodic interval.	Shall	Wi-Fi Communication
FR-18	Each transmitted message should include a device ID, sequence number, readings, status and fault flags.	Should	Wi-Fi Communication
FR-19	The system should continue local monitoring, display and alerting if Wi-Fi becomes unavailable.	Should	Main Loop / Wi-Fi
FR-20	The system should attempt Wi-Fi reconnection automatically, without blocking the monitoring cycle.	Should	Wi-Fi Communication
FR-21	The system shall detect and indicate sensor faults (disconnected or persistently invalid sensor).	Shall	Error Handler
FR-22	An OLED initialisation failure shall not prevent monitoring, alerting or transmission.	Shall	Error Handler / Display Manager
FR-23	Thresholds, sampling and transmission intervals should be held as named configuration values separate from logic.	Should	Configuration
FR-24	The system should output diagnostic messages to the serial port.	Should	Error Handler
FR-25	The system should use a watchdog timer to recover from firmware lock-up.	Should	Main Loop / Error Handler
FR-26	Alert outputs shall be OFF by default at reset and until the first valid evaluation.	Shall	System Init / Alert Manager

6. Non-Functional Requirements

Non-functional requirements describe qualities of the system rather than specific behaviours. The verification column states how each is expected to be checked; verification itself is planned for later weeks and has not been performed.

Table 2: Non-functional requirements

ID	Category	Requirement	Proposed target / verification approach
NFR-01	Reliability	The node shall run unattended and recover from transient faults (Wi-Fi loss, single failed sensor read) without manual reset.	Retry and reconnect logic, watchdog; planned extended run in Week 4.
NFR-02	Reliability	Failure of one subsystem (sensor, OLED, Wi-Fi) shall not stop the remaining functions.	Fault-injection checks (for example unplugging a sensor) in Weeks 3 to 4.
NFR-03	Maintainability	Source code shall be divided into modules with a single responsibility; pins, thresholds and timings shall be named constants in one configuration file.	Code review against the module list in Section 15.
NFR-04	Scalability	Adding a further sensor shall require a new acquisition driver and a new data field, not a redesign. Multiple nodes shall be distinguishable by device ID.	Design review; payload includes device_id.
NFR-05	Usability	OLED text shall be legible at about 0.5 to 1 m, with labelled units and plain-language status words rather than codes.	Visual inspection of the display layout (Section 8).
NFR-06	Performance	Sampling, processing and display update shall complete within their scheduled periods (Section 13) with ample processor margin.	Timing instrumentation in Week 3.
NFR-07	Responsiveness	An alert shall be raised within two sampling periods of a confirmed threshold crossing; network operations shall not delay it.	Timed alert test in Weeks 3 to 4.
NFR-08	Power efficiency	The node shall operate from a single 5 V USB supply with peak demand within a 1 A source and shall avoid unnecessary radio activity.	Supply sizing (Section 12); current measurement in a later week.
NFR-09	Modularity	Replacing a sensor type shall change only the corresponding driver module.	Interface definitions in Section 15.
NFR-10	Portability	Hardware access shall be isolated behind a thin layer so that the application logic can be moved to another ESP32 board or pin map with minimal change.	Central pin map; code review.
NFR-11	Safety	The design shall use only low-voltage (5 V and below) circuits, limit GPIO currents with resistors or a transistor driver, and default outputs to OFF. It shall not be presented as a certified gas or life-safety detector.	Hardware design review; documentation wording.
NFR-12	Security	Credentials shall not be stored in shared source code; transport should be encrypted; the node shall not accept unsolicited inbound connections.	Design review (Section 21).
NFR-13	Environmental	The node shall operate indoors in non-condensing conditions within the assumed operating range.	Assumption A-06; checked against datasheet ranges.

7. Input Specifications

7.1 Sensor Inputs

Quantities in the table below are typical published figures for the selected parts and are quoted only to size the design. They must be confirmed against the datasheets of the actual components.

Table 3: Sensor and system inputs

Input	Source	Signal type	Typical published range / notes	Proposed acquisition
Temperature	DHT22 (AM2302)	Digital, single-wire protocol	About -40 to +80 degrees C; typical accuracy about +/-0.5 degrees C; 0.1 degree C resolution.	Every 2 s (device minimum sampling period is about 2 s).

Input	Source	Signal type	Typical published range / notes	Proposed acquisition
Relative humidity	DHT22	Digital, single-wire protocol	0 to 100 %RH; typical accuracy about +/-2 to 5 %RH; 0.1 %RH resolution.	Every 2 s, together with temperature.
Air-quality (gas) signal	MQ-135 module, analog output (AO)	Analog voltage read by the 12-bit ADC	Output voltage rises with gas concentration. Provides a relative indication only. Heater requires 5 V and warm-up.	Sampled about every 1 s with oversampling and averaging.
Configuration values	Firmware configuration file	Compile-time constants (baseline)	Thresholds, intervals, Wi-Fi SSID/password, endpoint address, device ID.	Read at start-up.
Wi-Fi link status	ESP32 Wi-Fi stack	Software status / events	Connected, disconnected, signal strength (RSSI).	Polled each cycle / event-driven.
Time base	ESP32 internal timer	Software timer	Used for non-blocking scheduling and uptime.	Continuous.
Reset cause	ESP32 reset-reason register	Software status	Distinguishes power-on, brown-out and watchdog resets.	Read once at start-up.

7.2 MQ-135 Signal Conditioning

The MQ-135 module has a heater that is normally powered from 5 V, and its analog output can therefore swing above the 3.3 V that the ESP32 ADC pins tolerate. The AO signal shall not be connected directly to an ESP32 pin. The proposed solution is a resistive voltage divider (for example 10 kOhm in series with 15 kOhm to ground, giving a ratio of 0.6, so that a 5 V output maps to about 3.0 V) with a small capacitor (for example 100 nF) at the ADC pin to suppress noise. Final values will be chosen in Week 2 after checking the actual module's output behaviour and source impedance.

The ESP32 ADC is 12-bit (0 to 4095 counts) but is not perfectly linear, particularly near the ends of its range. The MQ-135 is read on an **ADC1** channel (GPIO34), because ADC2 channels cannot be used while Wi-Fi is active. Multiple samples are averaged to reduce noise.

The MQ-135 output depends on the sensing-element resistance, which changes with gas concentration, temperature and humidity. For this reason the design uses a **baseline-relative index**: the processed reading is compared with a clean-air baseline, and status categories (GOOD / MODERATE / POOR) are defined from that ratio. The boundaries are configurable and will be set after baseline observation in later weeks; no numerical boundaries are claimed in Week 1.

7.3 Configurable Thresholds

Table 4: Proposed default thresholds (configurable)

Parameter	Proposed default	Comment
High temperature	35 degrees C	Initial proposal for an indoor space; adjustable per application.
Low temperature	10 degrees C	Initial proposal; may be disabled.
High humidity	80 %RH	Initial proposal; relates to condensation and mould risk.
Low humidity	20 %RH	Initial proposal; relates to very dry air.
Air-quality status boundaries	To be defined (baseline-relative)	Set after clean-air baseline observation; not specified in Week 1.
Hysteresis	Return limit set inside trigger limit (for example 1 degree C or 2 %RH)	Prevents repeated switching near the threshold.
Confirmation count	2 consecutive samples	Rejects single spurious readings; adds at most one sample period of delay.

These values are initial design proposals chosen for illustration; they are not results of measurement or testing and can be replaced with values from applicable comfort or storage guidelines.

8. Output Specifications

8.1 OLED Display

The display is a 0.96-inch, 128 x 64 pixel monochrome OLED, typically based on the SSD1306 controller, connected over I2C. The layout below is a proposal; placeholders such as xx.x indicate where values appear and are not real data.

Table 5: Proposed OLED screen layout

Row	Content (example format)	Purpose
1	ENV MONITOR WiFi: OK / OFFLINE	Title and Wi-Fi status.
2	Temp : xx.x C	Temperature with unit.
3	Hum : xx.x %	Relative humidity with unit.
4	Air : GOOD / MODERATE / POOR / WARMING UP	Air-quality status category.
5	Alert: NONE / TEMP / HUM / AIR	Active alert source.
6	Message line (for example SENSOR FAULT, OLED OK)	Fault or status messages.

8.2 Alert Outputs

Output	Hardware	Proposed behaviour
Alert LED	LED with series resistor on a GPIO pin	Steady ON while any alert is active; slow blink for a sensor fault (FR-15). OFF in normal state.
Buzzer	Active buzzer driven by a transistor from a GPIO pin	Intermittent beeping while an alert is active; silent in normal state. A mute button is a possible optional addition.

8.3 Wi-Fi Telemetry Output

Data sent to the conceptual dashboard is a compact message, independent of the transport (HTTP(S) or MQTT). The field list below defines its content; no data values are implied.

Table 6: Conceptual telemetry message fields

Field	Type	Description
device_id	String	Unique node identifier, supporting multiple nodes.
seq	Unsigned integer	Message sequence number, to detect missed transmissions.
uptime_s	Unsigned integer	Seconds since start; a simple time reference when no real-time clock or network time is available.
temp_c	Float or null	Validated temperature; null if invalid.
hum_pct	Float or null	Validated relative humidity; null if invalid.
aq_raw	Integer	Averaged MQ-135 ADC value, for later calibration analysis.
aq_status	String	GOOD, MODERATE, POOR or WARMING_UP.
alert_flags	Bit field	Which alerts are active.
fault_flags	Bit field	Which faults are active (sensor, display, Wi-Fi).
wifi_rssi_dbm	Integer	Signal strength at transmission time.

8.4 Serial Diagnostic Output

The USB-serial interface provides timestamped diagnostic messages (start-up stages, validation failures, Wi-Fi events, alert changes). It supports development in Weeks 2 to 4 and is not a user-facing output.

9. Hardware Requirements

Table 7: Proposed hardware list

Item	Proposed part / specification	Qty	Role
Microcontroller board	ESP32 development board (ESP32-WROOM-32 class)	1	Processing, GPIO, ADC, I2C, Wi-Fi, USB-serial and 3.3 V regulation.
Temperature/humidity sensor	DHT22 (AM2302), with 10 kOhm pull-up on the data line if the module lacks one	1	Temperature and humidity measurement.
Air-quality sensor	MQ-135 gas sensor module with analog output (AO)	1	Prototype air-quality / gas-sensing input.
Display	0.96-inch 128 x 64 OLED, I2C interface (SSD1306 class)	1	Local display of readings and status.
Visual alert	LED (for example red, 3 mm or 5 mm)	1	Visual alert indicator.
Audible alert	Active buzzer (rated for 3.3 V or 5 V)	1	Audible alert.
Buzzer driver	General-purpose NPN transistor, 1 kOhm base resistor (flyback diode if the buzzer is electromagnetic)	1 set	Switches buzzer current without overloading the GPIO.
Resistors / capacitors	LED series resistor (about 220 Ohm); divider resistors (about 10 kOhm and 15 kOhm); 100 nF decoupling and filter capacitors; 10 to 100 uF bulk capacitor	As needed	Current limiting, level scaling, filtering and supply decoupling.
Power source	5 V USB adapter, preferably rated 1 A or more, with USB cable	1	Supply for the whole node.
Prototyping	Breadboard and jumper wires	1 set	Interconnection during prototyping.

9.1 Proposed Pin Allocation

GPIO numbers below follow common ESP32 development-board conventions and are proposals to be confirmed in Week 2. Pins that influence boot behaviour (GPIO0, 2, 12, 15) and the pins used by the USB-serial port (GPIO1, 3) are avoided.

Table 8: Proposed ESP32 pin map

Signal	ESP32 pin	Interface	Notes
DHT22 DATA	GPIO4	GPIO (single-wire)	Bidirectional; 10 kOhm pull-up to 3.3 V.
MQ-135 AO (via divider)	GPIO34 (ADC1_CH6)	ADC	Input-only pin on ADC1; usable while Wi-Fi is active.
OLED SDA	GPIO21	I2C	Default I2C data pin in the Arduino core.
OLED SCL	GPIO22	I2C	Default I2C clock pin in the Arduino core.
Alert LED	GPIO25	GPIO output	Via about 220 Ohm series resistor.
Buzzer driver	GPIO26	GPIO output	Drives transistor base through 1 kOhm.
Mute button (optional)	GPIO27	GPIO input	Optional; internal pull-up; not required by Week 1 requirements.
Power	3V3, 5V (VIN), GND	Power	3.3 V for DHT22 and OLED; 5 V for MQ-135; common ground.

9.2 Physical Design Considerations

- Place the DHT22 away from the MQ-135 heater, the regulator and the ESP32 module, all of which generate heat that would bias the temperature and humidity readings.
- Mount the MQ-135 with free airflow; an enclosed or stagnant position distorts its response.
- Keep wiring to sensors short, and use a common ground reference for all modules.
- Position the OLED, LED and buzzer so that they are visible and audible to users.

10. Software Requirements

Software requirements describe the development environment and the engineering rules that the Week 2 firmware must follow. Specific libraries are proposed and will be confirmed in Week 2 according to licence, memory use and availability.

Table 9: Software requirements

ID	Requirement
SWR-01	Firmware shall be written in C/C++ using the Arduino framework for ESP32, developed with the Arduino IDE or PlatformIO (assumption A-02).
SWR-02	The main loop shall use non-blocking, timestamp-based scheduling (for example <code>millis()</code>) rather than <code>long delay()</code> calls.
SWR-03	Each logical module (Section 15) shall be a separate source/header pair with a documented interface.
SWR-04	Pins, thresholds, intervals, timeouts and device ID shall be defined in a central configuration header.
SWR-05	Wi-Fi credentials and access tokens shall be kept in a separate file that is excluded from version control.
SWR-06	Proposed library functions: DHT sensor driver, SSD1306/graphics driver, Wi-Fi/TCP-IP stack, HTTP or MQTT client, JSON serialiser.
SWR-07	All network operations shall use explicit timeouts.
SWR-08	The firmware shall use the ESP32 task watchdog and read the reset reason at start-up.
SWR-09	Fixed-size buffers should be preferred to dynamic memory allocation in the periodic path to avoid heap fragmentation.
SWR-10	Serial logging shall support selectable verbosity for debugging.
SWR-11	Source code shall be kept under version control and commented to the level needed for hand-over.

Supporting tools are a serial monitor, a multimeter for hardware checks, and, if physical hardware is not available in Week 2, a circuit/firmware simulator. Which of these is used is not yet decided and is not assumed here.

11. Communication Interfaces

Table 10: Interface summary

Interface	ESP32 resource	Connected device	Purpose / key parameters
I2C	GPIO21 (SDA), GPIO22 (SCL)	0.96-inch OLED	Display communication; 3.3 V logic; typically 100 or 400 kHz.
GPIO (digital)	GPIO4; GPIO25; GPIO26	DHT22 data; LED; buzzer driver	Single-wire sensor protocol and alert output control.
ADC	GPIO34 (ADC1_CH6)	MQ-135 analog output (via divider)	12-bit analog acquisition of the gas-sensor voltage.
Wi-Fi	Built-in 2.4 GHz radio and antenna	Wireless access point / router	Station-mode link for transmitting data to the remote dashboard.
UART (USB-serial)	UART0 (GPIO1, GPIO3) via on-board bridge	Development computer	Programming and diagnostic logging only.
BLE	Built-in Bluetooth LE radio	None in baseline design	Available but not used; possible future use for provisioning.

11.1 I2C (OLED)

I2C is a two-wire, multi-device serial bus with a clock line (SCL) and a data line (SDA). The ESP32 acts as the bus master and the OLED as a slave. Both lines are open-drain and require pull-up resistors to 3.3 V (commonly about 4.7 kOhm); many OLED modules already include them, so the existing values should be checked to avoid unintentionally paralleling several low-value resistors. SSD1306 modules normally respond at address 0x3C, sometimes 0x3D. A full 128 x 64 monochrome frame is 1024 bytes, so a complete refresh is quick even at standard speed. Because I2C needs only two pins and supports more devices on the same bus, it also leaves room for future I2C sensors.

11.2 GPIO (LED, buzzer and DHT22 data)

GPIO pins operate at 3.3 V logic and can source only limited current, so loads are not connected directly where they could exceed that limit. The LED is driven through a series resistor. The buzzer is switched by a transistor, which carries the buzzer current from the supply rail while the GPIO only supplies a small base current. The DHT22 uses a proprietary single-wire protocol on a GPIO pin: the ESP32 pulls the line low to request a measurement and the sensor replies with 40 bits of data using pulse widths measured in tens of microseconds. This timing-sensitive exchange can occasionally fail, so the design includes validation and retry (Section 20).

11.3 ADC (MQ-135)

The ESP32 contains two 12-bit successive-approximation ADC units. ADC1 channels (GPIO32 to GPIO39) can be used at any time; ADC2 channels share hardware with the Wi-Fi driver and are unavailable while Wi-Fi is active. The MQ-135 is therefore connected to an ADC1 pin. The input attenuation setting determines the measurable voltage range (up to roughly 3.1 to 3.3 V at the highest attenuation), and the converter is somewhat non-linear near the range limits. The design uses a voltage divider so that the signal stays within range, a capacitor to stabilise the input, and averaging over multiple samples.

11.4 Wi-Fi (remote data transmission)

The ESP32 includes an IEEE 802.11 b/g/n radio for the 2.4 GHz band. In station mode the node joins an existing access point using WPA2 (or newer, depending on the SDK version) credentials, obtains an address by DHCP, and uses the TCP/IP stack to send data to a server. Connection is managed by events and status checks so that loss of the link is detected and reconnection can be attempted at increasing intervals. Wi-Fi is the only network interface used in the baseline design.

11.5 UART and BLE

The UART connection through the USB-serial converter on the development board is used for programming and logging. The BLE radio is not part of the baseline system; it is noted as an available capability for future provisioning or local access.

12. Power Management

12.1 Power Architecture

The node is powered from a 5 V USB input. After input protection and filtering (a bulk capacitor and decoupling capacitors, and a fuse or ferrite bead if included in the final design), the 5 V rail feeds the regulator that produces the 3.3 V system supply. On a typical ESP32 development board this regulator is a linear (LDO) device built into the board. The MQ-135 heater is powered directly from the 5 V rail, because the heater is specified for 5 V operation; the ESP32, DHT22 and OLED run from 3.3 V.

Table 11: Power rails and loads

Rail	Source	Loads
5 V (VIN / USB)	USB adapter via protection and filtering	MQ-135 module (heater and electronics), input to the 3.3 V regulator, buzzer (depending on its rating).
3.3 V	On-board LDO regulator	ESP32 module, DHT22, OLED, pull-up resistors, LED branch (through GPIO).
GND	USB ground	Common reference for all modules.

12.2 Preliminary Current Estimate

The table gives order-of-magnitude figures from typical published values, used only to size the supply. They are planning estimates, not measurements; actual current will be measured in a later week.

Table 12: Preliminary supply-sizing estimate (not measured)

Load	Rail	Estimated current	Remarks
ESP32 module	3.3 V	About 80 to 240 mA, with brief higher peaks	Higher during Wi-Fi transmission; lower when idle.
OLED	3.3 V	About 10 to 25 mA	Depends on the number of lit pixels and contrast.
DHT22	3.3 V	About 1 to 1.5 mA while measuring	Negligible in standby.
MQ-135 heater	5 V	About 150 mA	Largest steady load; continuous.
LED	3.3 V (GPIO)	About 5 to 10 mA	Only during alerts.
Buzzer	5 V or 3.3 V	About 10 to 30 mA	Only during alerts; depends on the buzzer type.
Total (worst case)	-	Roughly 0.45 to 0.55 A	Brief peaks above this are possible, so a 1 A adapter is recommended.

12.3 Power-Management Considerations

- **Supply capability.** A standard USB 2.0 port is limited to 500 mA, which is marginal against the worst case above. A 5 V adapter rated at 1 A or more is the proposed source.
- **Linear regulator heating.** If the 3.3 V rail carries about 0.3 A from a 5 V input, a linear regulator dissipates about $(5.0 - 3.3) \times 0.3$, or roughly 0.5 W, as heat. This is acceptable for a small board but means the regulator area should not be near the DHT22.
- **Wi-Fi current bursts.** Radio transmission causes short current peaks. Local decoupling capacitors (100 nF near ICs and a larger bulk capacitor at the input) keep the 3.3 V rail stable and reduce the risk of brown-out resets.
- **Brown-out and power interruption.** The ESP32 brown-out detector resets the device if the supply sags. After any reset, outputs start OFF, the reset reason is logged, and the MQ-135 warm-up is restarted (Section 20).
- **Efficiency measures within the baseline.** Wi-Fi is used periodically rather than continuously, the display and sampling rates are modest, and the code avoids busy-wait loops. Large savings are limited because the MQ-135 heater runs continuously, so battery operation and deep sleep are treated as future work (Section 25).
- **Ground and wiring.** A single common ground, short supply leads and separate return paths for high-current loads reduce noise on the analog (ADC) measurement.

13. Real-Time and Performance Requirements

The system is a **soft real-time** monitor. Environmental quantities change over seconds to minutes, so strict microsecond deadlines are not required; however, sampling, display and alert actions must occur at predictable periods, and network activity must not interfere with them. The values below are **proposed design targets**. None of them has been measured, and they will be checked against actual behaviour in Weeks 3 and 4.

Table 13: Proposed real-time and performance design targets

Parameter	Proposed design target	Rationale
DHT22 sampling period	2 s	The device's minimum sampling period is about 2 s; faster polling gives errors.
MQ-135 sampling period	1 s, with multiple-sample averaging	The gas signal is slow; averaging reduces ADC noise.
MQ-135 warm-up	Several minutes before status is reported (exact time to be set from the datasheet and observation)	The heater and sensing layer need time to stabilise; status shows WARMING UP meanwhile.
OLED refresh	About once per second, or when a value or status changes	Matches the sensor update rate; avoids needless bus traffic.

Parameter	Proposed design target	Rationale
Alert response	Within two sampling periods of a confirmed threshold crossing (about 4 s or less)	Includes two-sample confirmation; prompt for slow environmental changes.
Wi-Fi transmission interval	About every 15 to 30 s (configurable)	Periodic reporting suits the slow signals and limits radio use.
Wi-Fi reconnect attempts	Progressive back-off (for example 5 s up to 60 s)	Avoids continuous reconnection attempts and keeps the loop responsive.
Network operation timeout	A few seconds, never unbounded	Ensures a failed connection cannot block local alerting.
Watchdog timeout	About 10 s	Recovers from firmware lock-up while allowing for slow operations.
Main-loop blocking	No operation blocks the loop for longer than a few tens of milliseconds, except bounded network calls handled separately	Keeps display and alert timing predictable.

13.1 Analytical Timing Estimates

The following estimates come from simple calculations on published interface characteristics and are used only to show that the proposed periods are plausible. They are not measurements.

Activity	Estimated duration	Basis
DHT22 read	About 5 to 10 ms of bus activity	Start signal plus 40-bit response at tens of microseconds per bit.
MQ-135 averaged read	A few milliseconds up to tens of milliseconds	Several ADC conversions with short spacing.
OLED full-frame update	About 25 ms at 400 kHz	Roughly 1030 bytes x 9 bit-times per byte at 400 kHz.
Network transmission	Hundreds of milliseconds to seconds, variable	Depends on the network and server, which is why it needs a timeout and separate handling.

Sensing, processing and display together occupy a small fraction of a two-second period, so there is ample margin. The variable network time is the main scheduling risk, and the design addresses it by running transmission on a schedule independent of alerting, either through short bounded calls or a separate FreeRTOS task. The choice is made in Week 2.

13.2 Design Targets versus Measured Results

To avoid confusion in later reports, this document separates three categories: **specified targets** (the table above), **datasheet-based estimates** (Sections 7, 12 and 13.1) and **measured results** (none exist at present). Weeks 3 and 4 will add measured data only where it has actually been obtained.

14. Hardware Architecture

The hardware is organised into five subsystems around the ESP32. Each subsystem has a single clear role and connects to the microcontroller through one interface type.

Table 14: Hardware subsystems

Subsystem	Components	Function
Sensing	DHT22; MQ-135 with divider/filter	Convert environmental quantities into digital data (DHT22) and a conditioned analog voltage (MQ-135).
Processing	ESP32	Run firmware, acquire data, validate and process it, evaluate thresholds, manage outputs and communications.
Local output (HMI)	OLED; LED; buzzer with driver	Present readings and status and give immediate visual and audible alerts.
Communication	ESP32 Wi-Fi radio; access point	Transport data to the remote dashboard.

Subsystem	Components	Function
Power	USB 5 V input; protection; 3.3 V regulator; decoupling	Provide 3.3 V and 5 V rails and stable supply to all loads.

14.1 Interface and Driver Circuits (Proposed)

Table 15: Proposed supporting circuits

Circuit	Proposed design	Purpose
DHT22 pull-up	About 10 kOhm from data line to 3.3 V	Provides the idle-high level for the single-wire protocol.
MQ-135 level scaling	Divider of about 10 kOhm and 15 kOhm, 100 nF at the ADC pin	Keeps the 0 to 5 V analog output within the ADC input range and reduces noise.
LED driver	About 220 Ohm series resistor; at a typical LED forward voltage near 2 V this gives roughly 6 mA	Limits LED current to a safe level for a GPIO pin.
Buzzer driver	NPN transistor, about 1 kOhm base resistor, flyback diode for an electromagnetic buzzer	Switches the larger buzzer current without loading the GPIO.
Supply decoupling	100 nF at each IC supply; 10 to 100 uF bulk at the input	Suppresses supply noise and radio-current spikes.

All component values are starting proposals and will be confirmed against datasheets when the circuit is prepared in Week 2.

14.2 Voltage Levels

ESP32 GPIO pins are 3.3 V devices and are not 5 V tolerant. Every signal entering the ESP32 must therefore be at or below 3.3 V. The DHT22 and OLED are powered from 3.3 V and so present compatible logic levels. The only 5 V-domain signal is the MQ-135 analog output, which is scaled by the divider described above.

15. Software Architecture

The software is organised in three conceptual layers: an **application layer** (main control loop, threshold logic, state management), a **service layer** (data processing, display management, alert management, communication, error handling) and a **driver / hardware-access layer** (sensor drivers, I2C/OLED driver, GPIO and ADC access, Wi-Fi stack). Higher layers call lower layers but not the reverse, which keeps hardware details out of the application logic. This section describes responsibilities only; no firmware is presented.

Table 16: Logical software modules

Module	Responsibility	Inputs	Outputs	Notes
System Initialization	Start-up sequence: configure clocks, GPIO, ADC, I2C, serial, watchdog; initialise sensors, OLED and Wi-Fi; set safe output states.	Configuration; reset cause	Initialised hardware; status flags	Runs once. Failures are recorded, not fatal, where possible.
Sensor Acquisition	Read DHT22 and MQ-135 on their scheduled periods; manage MQ-135 warm-up timer.	Hardware pins; timers	Raw readings and read-success flags	Contains the only code that touches sensor hardware.
Sensor Data Processing	Validate (range, missing, rate of change), average/filter, convert units, classify air-quality status.	Raw readings	Validated sensor snapshot	Produces the single data record used by other modules.
Display Management	Format and refresh the OLED screen; show status and fault messages.	Sensor snapshot; system status	I2C display writes	Operates only if display is available.
Threshold Monitoring	Compare snapshot with configured limits using hysteresis and confirmation count.	Sensor snapshot; thresholds	Alert state (type and on/off)	Pure logic; no hardware access.
Alert Management	Drive LED and buzzer according to alert and fault states and patterns.	Alert and fault state	GPIO outputs	Non-blocking pattern timing.

Module	Responsibility	Inputs	Outputs	Notes
Wi-Fi Communication	Connect, monitor and reconnect Wi-Fi; build and send the telemetry message at its interval.	Snapshot; status; network config	Network messages; link status	Bounded timeouts; never blocks alerts.
Error Handling	Maintain fault flags, retry counters, watchdog servicing, serial diagnostics.	Failure events from all modules	Fault flags; log messages; recovery actions	Central point for fault policy.
Main Control Loop	Schedule modules by timestamps; coordinate the monitoring cycle.	Timers; module states	Calls to modules	Cooperative, non-blocking loop (see below).

15.1 Scheduling Approach

The baseline design uses a cooperative loop in which each module has a due time checked against the system timer. Each module performs a short action and returns, so no module holds the processor for long. This is simple to debug and suits the small number of periodic activities. As an alternative, the Wi-Fi module can run as a separate FreeRTOS task so that a slow network call cannot delay sensing or alerting; the ESP32 Arduino framework already runs on FreeRTOS, so this option is available if the cooperative approach is not sufficient. The decision is deferred to Week 2 and should be based on observed behaviour.

15.2 System State Model

Table 17: System states

State	Entry condition	Behaviour	Exit
INIT	Power-on or reset	Hardware initialised; outputs OFF; warm-up timer started.	Initialisation complete.
WARM-UP	After INIT	Temperature/humidity monitored; air-quality status shown as WARMING UP; no air-quality alerts.	Warm-up time elapsed.
NORMAL	Readings within limits	Periodic sensing, display and transmission; alert outputs OFF.	Threshold confirmed exceeded, or fault detected.
ALERT	Confirmed threshold exceeded	LED and buzzer active; alert type displayed; sensing and transmission continue.	Readings return inside hysteresis band.
DEGRADED	Sensor, display or other fault persists	Remaining functions continue; fault indicated on display, LED pattern and in telemetry.	Fault clears after successful reads.

Wi-Fi connectivity is treated as an independent condition (CONNECTED, CONNECTING, OFFLINE) that runs alongside the states above. This keeps loss of connectivity from changing the monitoring and alert behaviour.

15.3 Shared Data (Conceptual)

Data structure	Contents
Sensor snapshot	Temperature, humidity, averaged air-quality value, air-quality status, validity flags and a sample sequence number.
System status	Current state, active alerts, fault flags, Wi-Fi state, reset cause, uptime.
Configuration	Pin map, thresholds, hysteresis, intervals, timeouts, device ID, network settings.

16. Detailed Block Diagram

Figure 1 shows the complete hardware architecture. Interfaces are colour-coded: power (red), analog/ADC (orange), digital GPIO (green), I2C (blue) and Wi-Fi (purple). The dashboard is drawn with a dashed outline because it is conceptual and not implemented.

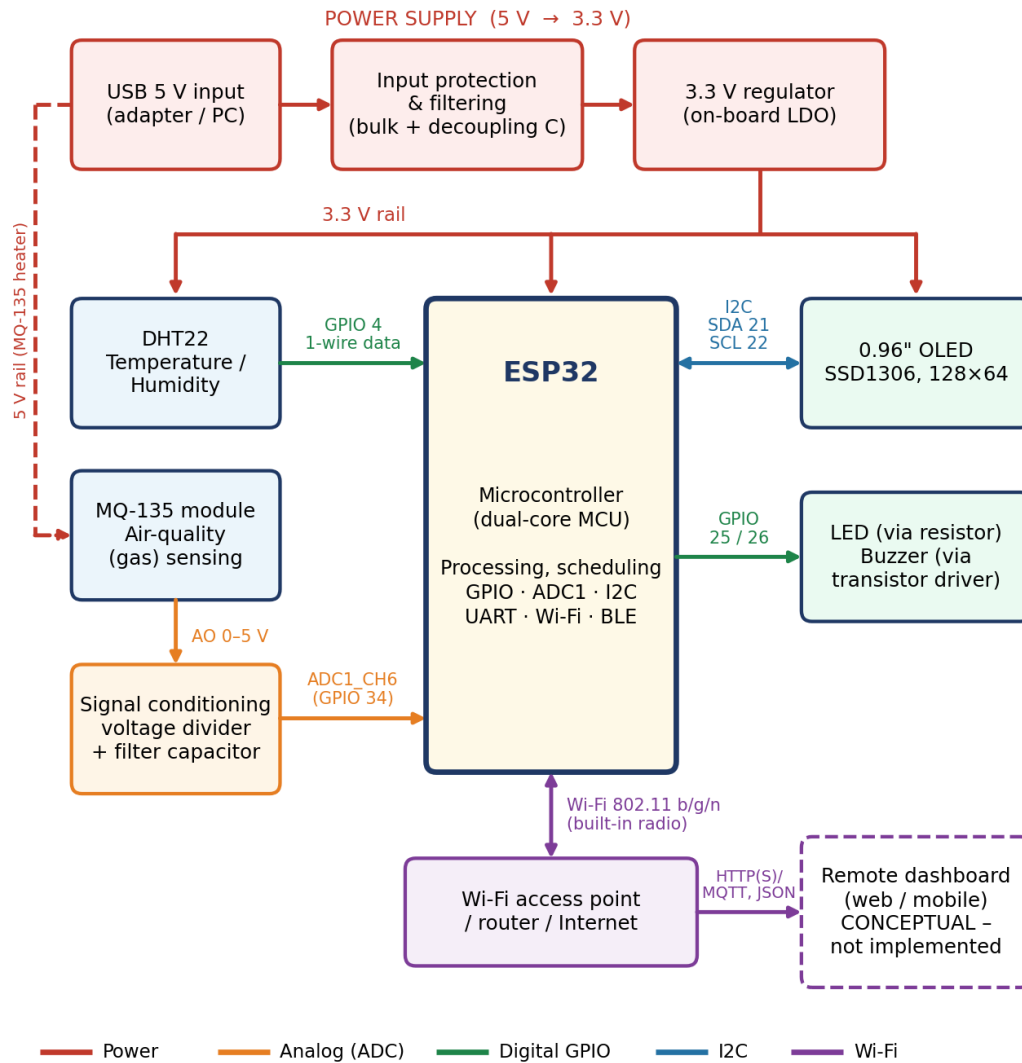


Figure 1: Detailed hardware block diagram of the proposed IoT-Based Smart Environmental Monitoring and Alert System

16.1 Explanation of Each Block

Block	Explanation
USB 5 V input	The single external supply. Also the source of the 5 V rail that powers the MQ-135 heater.
Input protection and filtering	Bulk and decoupling capacitors (and optional fuse or ferrite) that smooth the input and absorb current spikes.
3.3 V regulator	Converts 5 V to the 3.3 V supply for the ESP32, DHT22 and OLED (an on-board LDO in the baseline).
DHT22	Digital temperature and humidity sensor; one data wire to GPIO4.
MQ-135 module	Gas-sensing element with heater and on-board load resistor; produces an analog voltage that varies with gas concentration.
Signal conditioning	Voltage divider and filter capacitor that bring the MQ-135 output into the ADC range of GPIO34.
ESP32	Central controller: runs all software modules, reads sensors, drives outputs and handles Wi-Fi.
OLED display	Local display on I2C (GPIO21/22) of readings, alert and Wi-Fi status.
LED and buzzer	Local alert outputs driven from GPIO25 (LED) and GPIO26 (buzzer, via transistor).
Wi-Fi access point / Internet	Network path from the ESP32 radio to the remote server.
Remote dashboard (conceptual)	A web or mobile interface that would display the transmitted data. Not implemented in this project phase.

16.2 Diagram Specification (for Redrawing)

The table below lists every connection so that the diagram can be recreated in Word, PowerPoint, draw.io or another tool. Place the ESP32 centrally, power blocks along the top, sensors on the left, outputs on the right and the network path along the bottom, and use one colour per interface type.

Table 18: Connection specification

ID	Connection (source to destination)	Signal / interface	Direction	Voltage level
C-01	USB input to protection/filter	Power	One-way	5 V
C-02	Protection/filter to 3.3 V regulator	Power	One-way	5 V in
C-03	3.3 V regulator to ESP32 (3V3)	Power	One-way	3.3 V
C-04	3.3 V rail to DHT22 and OLED supply pins	Power	One-way	3.3 V
C-05	5 V rail to MQ-135 supply (heater)	Power	One-way	5 V
C-06	DHT22 DATA to ESP32 GPIO4	GPIO single-wire	Bidirectional	3.3 V, 10 kOhm pull-up
C-07	MQ-135 AO to divider to ESP32 GPIO34	ADC (analog)	To ESP32	0 to 5 V scaled to 3.0 V max
C-08	ESP32 GPIO21 and GPIO22 to OLED SDA and SCL	I2C	Bidirectional (SDA), master clock (SCL)	3.3 V
C-09	ESP32 GPIO25 to LED through series resistor	GPIO output	From ESP32	3.3 V
C-10	ESP32 GPIO26 to buzzer driver transistor	GPIO output	From ESP32	3.3 V
C-11	ESP32 radio to Wi-Fi access point	Wi-Fi 802.11 b/g/n	Bidirectional	Wireless
C-12	Access point / Internet to remote dashboard (conceptual)	HTTP(S) or MQTT, JSON	Towards dashboard	Not applicable
C-13	Common ground to all modules	Ground	Common	0 V

17. Software Flowchart

Figure 2 shows the operating flow of the firmware as proposed. It follows the required operating sequence, with the failure branches (display fault, Wi-Fi unavailable, invalid readings, transmission skipped) drawn explicitly so that it is clear that none of them stops the monitoring loop.

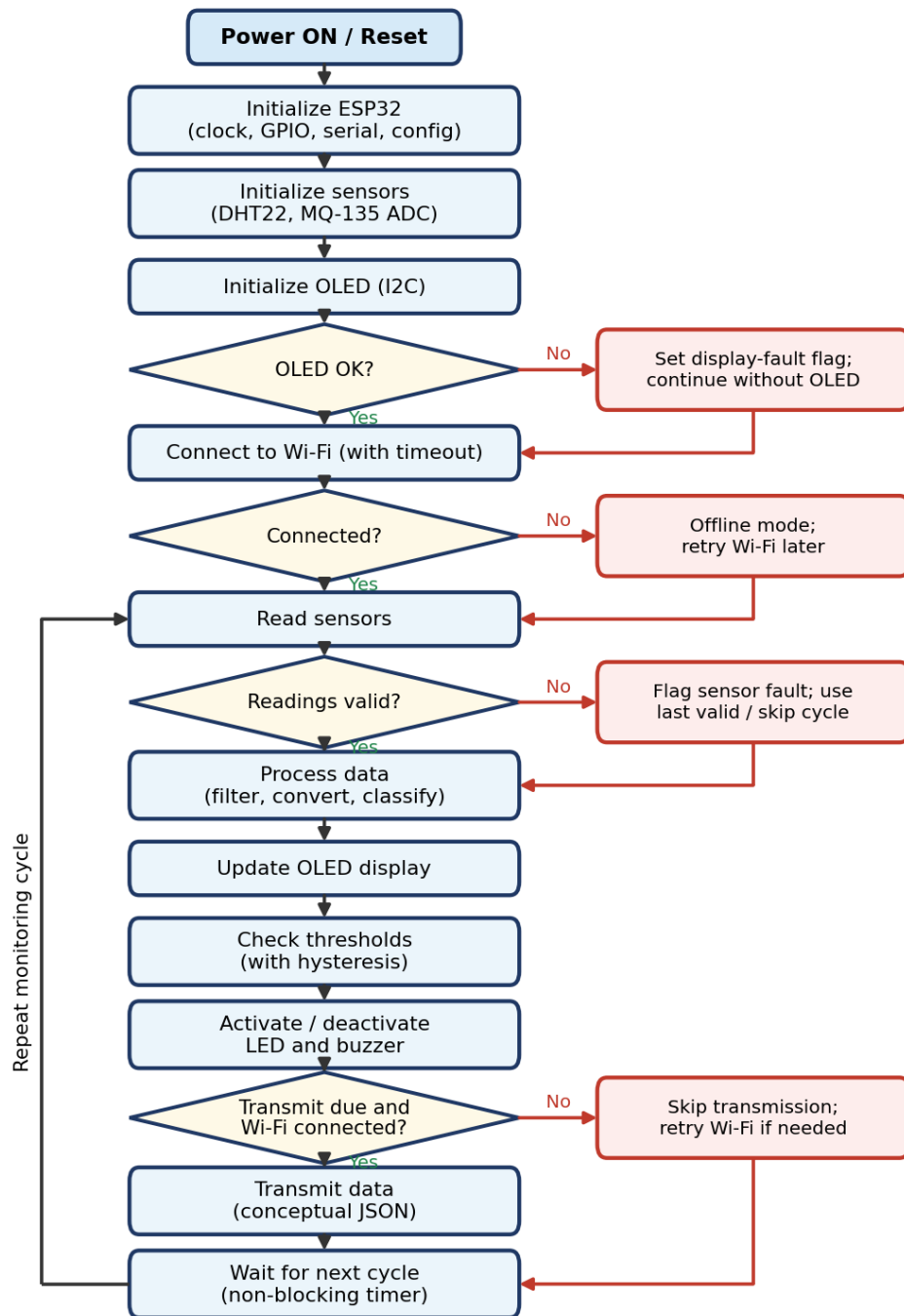


Figure 2: Software flowchart: start-up sequence and repeating monitoring cycle

17.1 Flowchart Notes

- **Start-up failures are non-fatal.** An OLED failure sets a display-fault flag; a Wi-Fi failure selects offline mode. In both cases the flow continues to the monitoring loop.
- **Invalid data are not used.** When a reading fails validation, a fault is flagged and the last valid value is held or the cycle is skipped, so alerts are never based on invalid data.
- **Alert evaluation is independent of the network.** Thresholds are checked and outputs updated before any transmission is attempted.
- **Transmission is conditional.** Data are sent only when the interval has elapsed and Wi-Fi is connected; otherwise reconnection is attempted in the background.

- **Non-blocking wait.** The wait step is a timer check rather than a long delay, so the loop remains responsive to alerts and reconnection.

18. Data Flow

Data move through the system in one direction: from sensing, through validation and processing, to local presentation, decision and network reporting. The chain is:

Environment > Sensors (DHT22, MQ-135) > ADC / GPIO acquisition > Validation > Processing and classification > [OLED display | Threshold evaluation > LED and buzzer | Telemetry message > Wi-Fi > Remote dashboard (conceptual)]

Table 19: Data items and their flow

Data item	Source	Format	Processing	Destination
Temperature	DHT22	Digital, 0.1 degree C resolution	Validity and range check; optional smoothing	OLED; threshold monitor; telemetry
Humidity	DHT22	Digital, 0.1 %RH resolution	Validity and range check	OLED; threshold monitor; telemetry
Air-quality raw value	MQ-135 via ADC	12-bit integer (0 to 4095)	Averaging; baseline-relative index; classification	OLED (status); threshold monitor; telemetry
Alert state	Threshold monitor	Flags and type	Hysteresis and confirmation	Alert manager; OLED; telemetry
Fault flags	All modules	Bit field	Aggregated by error handler	OLED; LED pattern; telemetry; serial log
Wi-Fi status	Wi-Fi stack	Enumeration, RSSI	Link monitoring	OLED; telemetry
Telemetry message	Wi-Fi module	JSON (conceptual)	Built from the sensor snapshot and status	Remote dashboard via access point

Conceptually, the remote path continues from the access point over the Internet to a server endpoint, a data store and a dashboard. These elements are outside the embedded device and are not implemented in the project; the device-side responsibility ends at the transmission of a correctly formed message.

19. Component Selection and Design Justification

19.1 Why ESP32?

The project needs analog input, digital I/O, an I2C bus, network connectivity and enough processing power to run several periodic tasks. The ESP32 provides all of these in one inexpensive device, without extra communication modules.

Table 20: ESP32 capabilities against project needs

Project need	ESP32 capability	Use in this design
Processing	Dual-core 32-bit microcontroller with FreeRTOS support	Runs sensing, processing, display and communication software.
Wi-Fi	Integrated 2.4 GHz 802.11 b/g/n	Transmission to the remote dashboard.
Bluetooth / BLE	Integrated Bluetooth and BLE	Not used initially; available for future provisioning.
GPIO	Many configurable general-purpose pins	DHT22 data, LED, buzzer driver.
ADC	12-bit ADC with multiple channels	MQ-135 analog input.
I2C, SPI, UART	Hardware peripherals for all three	I2C for the OLED; UART for debug; SPI available for future devices.
Low-power capabilities	Sleep modes and modem power saving	Not used in the baseline; supports the future battery option.

Table 21: Alternatives considered (qualitative)

Alternative	Advantages	Reason it is less suitable here
Arduino Uno class (ATmega328P)	Simple, widely documented, 5 V logic.	No built-in Wi-Fi; limited memory; would need an extra network module.
ESP8266	Low cost, built-in Wi-Fi.	Only one analog input, fewer pins and no BLE; less headroom.
STM32 family	Powerful, rich peripherals, strong low-power options.	No built-in Wi-Fi in common parts; larger setup effort for a four-week task.
Raspberry Pi Pico W	Wi-Fi, good documentation.	Comparable, but the ESP32 offers a broader ecosystem for IoT and BLE; selection is a design choice rather than a strict necessity.

19.2 Why DHT22?

The DHT22 measures both temperature and humidity through a single digital pin, which keeps wiring and ADC usage minimal. Compared with the DHT11 it has a wider range and better resolution and accuracy. It is inexpensive, widely supported by libraries and sufficient for room-level monitoring. Its slow update rate (about one reading per two seconds) matches the slow dynamics of room conditions. Higher-accuracy I2C sensors such as the SHT3x or BME280 families are better technically but cost more; they are retained as an upgrade option (Section 25).

19.3 Why MQ-135?

The MQ-135 is a low-cost, widely available gas sensor whose analog output varies with the presence of gases such as ammonia, nitrogen oxides, alcohol, benzene and smoke, and with CO₂-related air contaminants. It allows the project to demonstrate analog acquisition through the ESP32 ADC, signal conditioning and baseline-relative classification at low cost. It is chosen as a **prototype air-quality indicator**, not as an accurate analyser: it is non-selective (it responds to many gases without identifying them), needs warm-up and calibration, and is affected by temperature and humidity. These limits are treated as design constraints in Sections 7, 20 and 24.

19.4 Why an I2C OLED?

A 0.96-inch OLED is compact, has high contrast without a backlight, draws little current and needs only two signal pins plus power. This conserves GPIO pins and fits the I2C bus of the ESP32. A character LCD would use more pins or need an I2C adapter, and is larger and less flexible for status text and simple graphics.

19.5 Why Wi-Fi?

Wi-Fi is built into the ESP32, so no extra hardware is needed, and it connects directly to existing local networks and the Internet. It supports standard protocols (HTTP, MQTT) with simple client libraries. Alternatives such as LoRa, Zigbee or cellular could reach farther or use less power but require extra modules, gateways or subscriptions, which is not justified for an indoor, powered node. The main trade-off is dependence on network availability and higher power use, which the design handles by continuing local operation when offline and by transmitting periodically.

19.6 Why LED and Buzzer?

Alerts must be noticeable without a network or a screen. An LED provides a clear visual indication and a buzzer provides an audible one, so an alert can still be noticed if the person is not looking at the device. Both are inexpensive, need almost no software, and work independently of Wi-Fi.

19.7 Why a Modular Hardware and Software Architecture?

The system has several loosely related functions with different timing requirements and failure modes. Separating them into hardware subsystems and software modules with defined interfaces keeps faults contained (a failed display does not stop alerts), lets modules be developed and tested one at a time in Weeks 2 and 3, makes it

possible to replace a sensor without changing application logic, and gives a clear route to future features such as additional sensors or cloud integration.

20. Error Handling and Fault Considerations

The guiding principle is that faults should be **detected, contained and made visible**: the system should keep doing everything it still can, and should never present invalid data as valid. Counts and timings shown below are proposals.

Table 22: Fault analysis and handling strategy

Fault	Detection	Handling strategy	Indication
Sensor disconnected (DHT22)	Read returns no response, timeout or not-a-number; repeated failures	Retry a few times; after about three consecutive failures declare a sensor fault; stop using DHT22 data for alerts; keep other functions running; keep retrying periodically so recovery is automatic.	Fault message on OLED; LED fault pattern; fault flag in telemetry.
Sensor disconnected (MQ-135)	ADC value stuck at an extreme or implausibly constant, or unrealistic against the baseline	Treat as invalid, flag a fault, suspend air-quality alerts, continue other monitoring.	As above.
Invalid sensor readings	Range check against sensor limits; rate-of-change check	Reject the sample; keep the last valid value for a limited time or skip the cycle; never alert on a rejected sample.	Serial log; fault flag if persistent.
Unexpected values / noise spikes	Outlier check and comparison with recent samples	Median or moving-average filtering; require two consecutive samples to confirm a threshold crossing.	None unless persistent.
Wi-Fi unavailable	Connection timeout; disconnect event; failed send	Enter offline mode; continue local monitoring and alerts; retry with progressive back-off; resume transmission when connected.	Wi-Fi: OFFLINE on OLED; flag in telemetry once reconnected.
Transmission failure	Timeout or error response	Abort within the timeout; skip this report; try again at the next interval; optionally keep a small buffer of recent messages (future).	Serial log.
OLED initialisation failure	No acknowledge on I2C at expected address	Record display-fault flag; continue without display; optionally retry later; alert and Wi-Fi functions unaffected.	LED fault pattern; flag in telemetry; serial message.
Power interruption or brown-out	Reset-reason register at start-up; brown-out detector	Restart from the safe state: outputs OFF, full initialisation, warm-up timer restarted; log the reset cause.	Serial log; optional message on OLED.
Firmware lock-up	Task watchdog not serviced	Automatic reset and restart.	Reset cause logged.

Fault flags are held in a single bit field maintained by the error-handling module. Because they are included in telemetry and in the display, a remote user and a local user can both see when the node is degraded.

21. Reliability and Security Considerations

21.1 Reliability

- **Isolation of faults.** Each subsystem failure is handled by the error-handling module so that others continue (NFR-02).
- **Bounded operations.** Timeouts on all network operations and non-blocking timing keep the main loop deterministic.
- **Recovery.** A watchdog, brown-out detection and automatic Wi-Fi reconnection let the node recover without human intervention.
- **Stable alerting.** Hysteresis and confirmation prevent repeated on/off switching near thresholds.
- **Sensor drift.** MQ-135 baselines drift with age and conditions, so periodic re-calibration against clean air should be planned as a maintenance activity.

- **Power quality.** Decoupling, a suitably rated supply and short leads reduce resets and noisy ADC readings.

21.2 Security

The node is a small connected device on a local network, so basic security measures are included in the design. They are design-level intentions, and none has been implemented.

Table 23: Security considerations

Concern	Proposed measure	Status
Network access	Use a WPA2-protected (or stronger) network; avoid open Wi-Fi.	Design intention
Credential exposure	Keep Wi-Fi credentials and tokens in a separate file excluded from version control; do not print them in logs.	Design intention
Data in transit	Prefer encrypted transport (HTTPS or MQTT over TLS) with server certificate checking where memory and time permit.	Design intention; protocol chosen in Week 2
Unauthorised data submission	Use an API key or token for the server endpoint.	Conceptual
Inbound attacks	The device initiates outbound connections only and offers no open service ports.	Design intention
Physical access	Serial and programming access is possible with physical access to the board; this is accepted for a prototype.	Accepted risk
Firmware updates	Secure over-the-air updates are not included; they are future work (Section 25).	Future work

21.3 Safety Note

The system is low-voltage and uses current-limited outputs. However, it relies on a low-cost gas sensor that has not been calibrated or certified. The air-quality indication must therefore be regarded as informational, and the node must not be used as a substitute for certified carbon-monoxide, combustible-gas or other life-safety detectors.

22. Requirement Traceability Matrix

The matrix links requirements to the hardware component or software module that satisfies them and explains how. It will be extended in Weeks 2 to 4 with implementation and test references.

Table 24: Requirement traceability matrix

Requirement	Component / module	How the requirement is satisfied
FR-01 Hardware initialisation	System Initialization; ESP32	A defined start-up sequence configures GPIO, ADC, I2C, serial, watchdog, sensors, OLED and Wi-Fi, with outputs held OFF.
FR-02, FR-03 Temperature and humidity acquisition	DHT22; Sensor Acquisition; GPIO4	The DHT22 is read over its single-wire interface every 2 s and the result is passed to the processing module.
FR-04 Air-quality acquisition	MQ-135; divider; ADC1 (GPIO34); Sensor Acquisition	The conditioned analog voltage is sampled on ADC1 and averaged every 1 s.
FR-05, FR-06 Processing and validation	Sensor Data Processing	Range and plausibility checks, filtering, conversion and classification produce a validated snapshot.
FR-07 MQ-135 warm-up	Sensor Acquisition; Threshold Monitoring; WARM-UP state	A warm-up timer suppresses air-quality alerts and shows WARMING UP until stable.
FR-08 to FR-10 Display	OLED; Display Management; I2C	The module draws temperature, humidity, air status, alert and Wi-Fi status on the 128 x 64 display via I2C.
FR-11, FR-12 Thresholds and hysteresis	Threshold Monitoring; Configuration	Limits and hysteresis values are configuration constants; confirmation over consecutive samples avoids chatter.
FR-13, FR-14 Alert activation and clearing	LED; buzzer; Alert Management; GPIO25, GPIO26	Confirmed alert states switch the LED and buzzer drivers on; return inside the hysteresis band switches them off.

Requirement	Component / module	How the requirement is satisfied
FR-16, FR-17 Wi-Fi link and transmission	ESP32 Wi-Fi; Wi-Fi Communication	The module joins the configured network and sends the telemetry message at the configured interval.
FR-19, FR-20 Offline operation and reconnection	Main Control Loop; Wi-Fi Communication	Wi-Fi is independent of sensing and alerting; reconnection uses back-off and does not block the loop.
FR-21, FR-22 Fault detection and OLED failure tolerance	Error Handling; Display Management	Retry counters and flags detect faults; the display module is bypassed if the OLED is absent.
FR-25 Watchdog recovery	Error Handling; Main Control Loop	The task watchdog resets the device if the loop stops servicing it.
NFR-03, NFR-09, NFR-10 Maintainability, modularity, portability	All modules; central configuration	Single-responsibility modules, driver layer and one pin map isolate hardware dependencies.
NFR-07 Responsiveness	Main Control Loop; Threshold Monitoring; Wi-Fi Communication	Non-blocking scheduling and bounded timeouts keep alert latency within two sampling periods.
NFR-08 Power efficiency / supply	USB supply; regulator; Wi-Fi Communication	Supply sized for the estimated peak demand; periodic rather than continuous transmission.
NFR-11 Safety	Driver circuits; Alert Management; documentation	Current-limited outputs, low-voltage design, OFF by default and a clear non-certified use statement.
NFR-12 Security	Wi-Fi Communication; configuration handling	Separate credential storage, encrypted transport preferred, outbound-only connections.

23. Methodology

Week 1 followed a structured, top-down systems-engineering approach: define what the system must do, decide how it will be built, and record the decisions in a form that can be checked later. Information about component characteristics comes from typical published manufacturer information and ESP32 platform documentation, to be verified against the specific datasheets used; no experimental data was produced.

Table 25: Week 1 methodology

No.	Activity	Description	Output
1	Application analysis	Identified who uses the system, in which settings, and what they need to know and do.	Sections 1 to 4
2	Requirements elicitation	Converted the needs into numbered functional and non-functional requirements and assumptions.	Sections 4.5, 5, 6
3	Interface specification	Defined each input, output and communication interface, signal levels and constraints (for example ADC1 versus ADC2).	Sections 7, 8, 11
4	Component selection	Compared candidate parts against requirements and recorded the reasons for each choice.	Sections 9, 19
5	Power analysis	Assigned supply rails, estimated loads and checked supply and regulator margins.	Section 12
6	Timing analysis	Set design targets for sampling, display, alerts and transmission and estimated interface timing.	Section 13
7	Architecture design	Partitioned hardware into subsystems and software into modules, and drew block diagram, flowchart and state model.	Sections 14 to 18
8	Fault and risk analysis	Considered how each subsystem could fail, and defined handling, reliability and security measures.	Sections 20, 21
9	Traceability and review	Linked requirements to components and modules and reviewed the document for consistency and honest scope.	Sections 22, 24

Throughout, claims were restricted to what a design document can legitimately state. Where a value could not be justified from published typical figures or simple calculation, it was left to be determined (for example the air-quality category boundaries).

24. Limitations

Table 26: Known limitations of the proposed design

Area	Limitation	Design response
MQ-135 accuracy	Not a laboratory-grade analyser. It responds to several gases without identifying them (poor selectivity); its output varies with temperature and humidity; it needs a warm-up period and calibration; individual sensors differ; and it drifts with age. It cannot give a reliable ppm value without calibration against a reference.	Use as a relative, qualitative indicator with a clean-air baseline; show WARMING UP; document the limitation; list a better sensor as future work.
DHT22 performance	Slow update (about one reading per 2 s), moderate accuracy, possible read failures, self-heating and drift in prolonged high humidity, and sensitivity to nearby heat sources.	Sample at 2 s, validate and retry, place away from heat, and consider an upgrade later.
Wi-Fi dependency	Remote reporting requires an available network, and Wi-Fi increases power use.	Local monitoring and alerts continue offline; periodic transmission; automatic reconnection.
ESP32 ADC	Non-linear and noisy; ADC2 is unavailable with Wi-Fi.	Use ADC1, averaging and a filter capacitor.
Prototype / simulation nature	The design has not been built or tested; all figures are design targets or estimates; the dashboard is conceptual.	State this clearly; verification is planned for Weeks 2 to 4.
Calibration	Thresholds and air-quality categories are not yet calibrated.	Defer boundaries until a baseline is observed; keep values configurable.
Environmental variability	Room layout, airflow, sunlight, cooking, cleaning products and occupancy affect readings.	Use hysteresis and averaging; document placement guidance.
Data persistence	No local data logging is planned; data may be lost during Wi-Fi outages.	Future buffering or local storage.
Power	Continuous MQ-135 heater operation prevents battery use and deep sleep.	Mains/USB supply assumed; battery operation is future work.

25. Future Improvements

The items below are **future work and are not part of the current design or implementation**.

Table 27: Possible future improvements (not implemented)

Improvement	Description and benefit	Status
Cloud database	Store readings in a server-side database for long-term records and analysis.	Future work
Mobile application	A phone interface for viewing current readings and receiving notifications.	Future work
Historical graphs	Charts of trends over hours and days on the dashboard.	Future work
Better air-quality sensor	A digital sensor with better selectivity and defined outputs (for example a CO ₂ or VOC sensor on I2C).	Future work
Battery operation	Run from a battery pack, for portability.	Future work
Solar charging	Extend battery life with a small solar panel and charge controller.	Future work
Deep-sleep modes	Reduce average current by sleeping between measurements; limited by MQ-135 heater needs unless the sensor is replaced.	Future work
OTA firmware updates	Update firmware over Wi-Fi with authentication and verification.	Future work
More environmental sensors	Pressure, light, particulate matter or additional gases.	Future work
Advanced anomaly detection	Trend-based or statistical detection of unusual changes beyond fixed thresholds.	Future work

Improvement	Description and benefit	Status
Persistent configuration	Change thresholds and network settings at run time and store them in non-volatile memory.	Future work

26. Week 1 Deliverables

26.1 Deliverables Produced

No.	Deliverable	Where
1	Problem statement, objectives and system overview with assumptions	Sections 2 to 4
2	Functional and non-functional requirements (numbered)	Sections 5, 6
3	Input, output, hardware, software and interface specifications	Sections 7 to 11
4	Power architecture and preliminary supply estimate	Section 12
5	Real-time and performance design targets	Section 13
6	Hardware architecture, pin map and detailed block diagram	Sections 9, 14, 16
7	Software architecture, state model and software flowchart	Sections 15, 17
8	Data flow description	Section 18
9	Component selection and design justification	Section 19
10	Fault, reliability and security analysis	Sections 20, 21
11	Requirement traceability matrix	Section 22
12	Methodology, limitations and future improvements	Sections 23 to 25

26.2 Foundation for the Following Weeks

The following weeks are planned work that builds on this document. None of them is reported here as completed.

Table 28: Planned use of Week 1 outputs in later weeks

Week	Planned focus	Week 1 outputs it builds on
Week 2: Firmware Development and Implementation	Implement the software modules and drivers and verify them step by step.	Requirements, pin map, module definitions, state model and flowchart.
Week 3: Debugging and Performance Optimization	Debug, measure timing and current, tune filtering and thresholds.	Real-time targets, fault handling table and power estimates.
Week 4: System Integration and Final Testing	Integrate all parts, test against requirements and document results.	Traceability matrix and non-functional requirements.

27. Conclusion

This report has defined the requirements and architecture of an ESP32-based IoT environmental monitoring and alert system. It describes a practical application, sets out 26 functional and 13 non-functional requirements, specifies inputs, outputs and interfaces, and presents a block diagram, software flowchart and state model. The design addresses the main constraints directly: the MQ-135 is limited to a relative indicator with signal conditioning, warm-up and a baseline; the ADC choice avoids a conflict with Wi-Fi; the power architecture accounts for the 5 V heater load and the current peaks of the radio; and the software is modular and non-blocking so that sensing and alerting continue when Wi-Fi fails.

The report also records what has not been done. No hardware has been assembled or tested, no firmware has been written, no measurements exist and no cloud service has been deployed. Performance figures are design targets and datasheet-based estimates, and component values are proposals to be confirmed.

The architecture, requirements and traceability matrix produced in Week 1 become the foundation for the rest of the internship: **Week 2 (Firmware Development and Implementation)** will implement the modules defined here; **Week 3 (Debugging and Performance Optimization)** will verify and tune timing, power and thresholds against the

targets; and **Week 4 (System Integration and Final Testing)** will integrate the subsystems and test the complete system against this requirement set.